BEST PRACTICES FOR PAGER ROTATION DUTIES IN DEVOPS

Carolina Rodriguez

CSD-380

Assignment 7.2

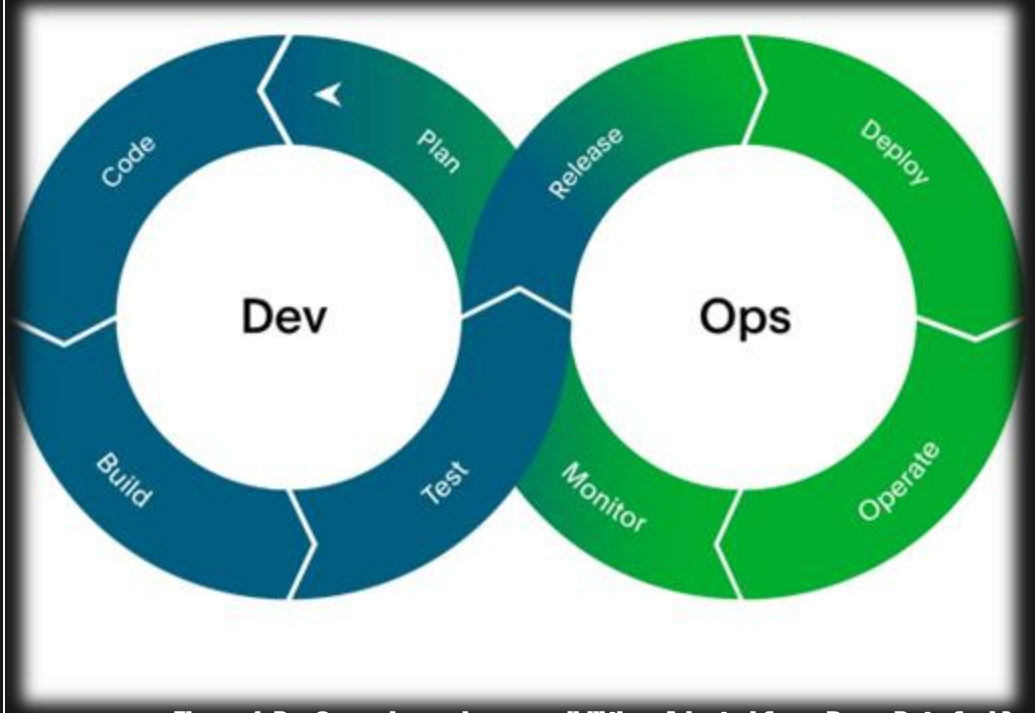

Figure 1. DevOps roles and responsibilities. Adapted from PagerDuty (n.d.).

WHAT PAGER ROTATION MEANS?

On-call engineers respond to production alerts, outages, and urgent service issues to keep systems stable.

WHY IT MATTERS IN DEVOPS?

Pager rotation duties involve monitoring system health, responding to alerts, using runbooks to diagnose and resolve issues, and escalating unresolved incidents to backup support when needed.

BEST PRACTICE #1: FAIR ROTATION

- A fair pager rotation helps prevent burnout and keeps responsibility shared across the team. No one person should always be responsible for nights, weekends, or high-stress shifts.
- A good DevOps rotation should include a **primary on-call person** and a **secondary backup**.
- The primary person responds first, while the backup is available if the issue is too complex, the primary person misses the alert, or extra help is needed.
- Also, Atlassian recommends reviewing on-call reports to check how often team members are paged or woken up and adjusting schedules to keep the workload fair.

BEST PRACTICE #2: ACTIONABLE ALERTS

- Google's SRE guidance explains alerts should be sent quickly, focus on important user-facing features, and be based on issues users are experiencing, such as:
  - slow response times,
  - failed transactions,
  - service outages.
- Alerts should also include enough information for the on-call person to know what happened, where to look, and what steps to take next. If too many alerts don't require action, it can lead to alert fatigue.
- For example, an alert saying, "checkout page is failing for users" is more useful than a vague alert saying, "CPU usage is high."

BEST PRACTICE #3: RUNBOOKS & ESCALATION

- Runbooks are step-by-step guides that help the on-call person troubleshoot common issues. They should explain what the alert means, where to check logs or dashboards, possible causes, and what steps to take first.
- Escalation paths are important because the on-call person may not always be able to resolve an incident alone. 3 types of escalations can look like:
  - **Hierarchical escalations** where the issue is passed to someone with more experience or seniority.
  - **Functional escalations** where the issue is passed to the team or person with the right system knowledge, even if they are not more senior.
  - Or an **automatic escalation** when a tool sends the alert to a backup person if the first on-call person does not respond.

BEST PRACTICE #4: REVIEW & IMPROVE

- After an incident, the team should review:
  - what happened?
  - what went well?
  - what caused delays?
  - what can be improved?
- We can also call this good postmortem.
- The goal is not to blame the person on call, but to improve the alerts, documentation, and response process.
- Every incident should become feedback for improvement.

RESOURCES:

- Atlassian. (n.d.). *Escalation policies for effective incident management*. https://www.atlassian.com/incident-management/on-call/escalation-policies
- Google. (n.d.). *Incident management guide*. Google Site Reliability Engineering. https://sre.google/resources/practices-and-processes/incident-management-guide/
- OpenAI. (2026). *ChatGPT* .[Large language model]. https://chatgpt.com/
- PagerDuty. (n.d.). *On-call rotations and schedules*. https://www.pagerduty.com/resources/incident-management-response/learn/call-rotations-schedules/
- PagerDuty. (n.d.). *6 essential DevOps roles you need on your team*. https://www.pagerduty.com/resources/devops/learn/essential-devops-roles/